



Desafío Final: Implementación y ejecución del juego de una invasión Alién en el espacio a partir de C++

Integrantes del grupo: Clayder Areiza Pino & Arlington Zahir Llerena

CC: 10421450543 & CC: 72 277 711

Email: clayder.areiza@udea.edu.co, zahir.llerena@udea.edu.co

Número telefónico: + 57 300 726 22 82

Profesores: AUGUSTO ENRIQUE SALAZAR JIMENEZ & KEVIN DAVID MARTINEZ ZAPATA

Grupo 003. Código: 2570201 – Informática II -

Facultad de Ingeniería en Telecomunicaciones Región – Yarumal & Carmen del Viboral -

Universidad de Antioquia

4 de mayo de 2026

Objetivos técnico general

- **Diseñar y construir un motor de juego bidimensional (por el momento con características 2D, con vista cenital) basado en el paradigma de Programación Orientada a Objetos (POO), que integre un sistema de control en tiempo real para el usuario y una arquitectura de gestión de entidades múltiples (imágenes de naves, obstáculos espaciales, proyectiles y elementos de diseño para emular el espacio), garantizando la eficiencia en el procesamiento de eventos, la actualización de estados cinemáticos y la resolución de colisiones mediante lógica booleana en C++.**

Objetivos específicos

Para alcanzar el objetivo anterior, pretendemos dividir el proyecto en diversas fases que en nuestra opinión permiten una estructura adecuada de la iniciativa, más aún cuando debemos ser flexibles en la propuesta de código puesto que pretendemos apelar a clases y funciones modulares que se complementen entre sí. Este diseño modular, brinda muchas ventajas en términos de edición puesto que, si durante el proceso de construcción y

ejecución de estos módulos se detecta tareas inadecuada por parte de estos fragmentos de código, se puede editar el módulo correspondiente, sin afectar la integridad de la totalidad del algoritmo. Ante lo anterior y bajo la propuesta de diseño inicial podemos mencionar algunos objetivos preliminares que dentro de nuestra propuesta consideramos sumamente importantes:

- **Arquitectura de control y sincronización:** Establecer el bucle principal de ejecución y una clase de configuración centralizada para gestionar la sincronización de cuadros y las constantes globales del sistema. Pretendemos contar con una función main() que invoque clases y funciones modulares que realizaran diversas tareas, entre las cuales podemos destacar: despliegue del escenario o telón del espacio, función que nos habilite el movimiento de una nave espacial, función que habilite un conjunto de naves invasoras, etc.
- **Gestión dinámica de proyectiles y memoria:** Implementar contenedores de objetos definido a partir de los atributos que brinda C++ para proyectiles que incluyan rutinas de limpieza automática de instancias situadas fuera del dominio espacial para optimizar el uso de recursos.
- **Algoritmos de generación de flotas:** Diseñar y calcular una matriz bidimensional de entidades (naves enemigas) mediante lógica basada en punteros lo cual nos permitiría gestionar la capacidad espacial de la interfaz y los márgenes que delimitan el telón del espacio.
- **Cinemática colectiva y condiciones de frontera:** Programar el comportamiento de traslación de la flota utilizando movimiento de vectores de dirección y desplazamientos en el eje de ordenadas al detectar los límites del plano. Probablemente estemos obligados a utilizar punteros o pasos de referencia junto con otras herramientas de gestión de memoria de objetos para hacer eficiente la gestión de la información guardada en cada objeto.



ESTRUCTURA PRELIMINAR DE LA PROPUESTA

El desarrollo del proyecto Alién Invasión se fundamenta en la integración de diferentes bloques de código, cada uno de ellos definidos por programación orientada a objetos y la gestión de estados en tiempo real. Mediante la implementación de un motor de ejecución cíclico, el sistema coordina la ubicación de los diversos gráficos, el procesamiento de entradas y la actualización de coordenadas espaciales para múltiples elementos.

Este marco de trabajo permite que cada componente desde el proyectil individual hasta la flota automatizada funcione como una unidad autónoma dentro de un entorno controlado (Por lo menos como grupo pretendemos eso), logrando una simulación interactiva donde la lógica matemática define el comportamiento y la emulación del sistema global.

ARQUITECTURA DE CODIGO PARA LA PROPUESTA

El juego Alien Invasión, el grupo de trabajo lo ve como una oportunidad para aplicar el paradigma orientado a objetos, el manejo de estados y la gestión de ciclos de vida del algoritmo.

A continuación, el grupo de trabajo presenta un desglose analítico de la arquitectura del juego.

El diseño sigue una estrategia de “Separación de Responsabilidades” por llamarlo de alguna manera. En lugar de un archivo monolítico, el proyecto se fragmenta en módulos especializados.

Clase	Modulo	Responsabilidad
AlienInvasion	alien_invasion.cpp	Clase controladora (Ejecutor). Gestiona el ciclo de vida, los recursos y el flujo principal.
Settings	settings.cpp	Repositorio centralizado de configuración inicial y chequeo de las clases.
Ship	ship.cpp	Gestiona el comportamiento, posición y dibujo de los objetos disponibles para el jugador, para el caso: naves, artillería y algunos elementos del escenario.
Alien	alien.cpp	Representación de un enemigo individual y su lógica de movimiento.
Bullet	bullet.cpp	Comportamiento de los proyectiles disparados por el jugador.
GameStats	game_stats.cpp	Seguimiento de variables de estado (puntuación, vidas, nivel, estado de actividad).
Button	button.cpp	Gestión de la interfaz de usuario (Botón de inicio).

Scoreboard	scoreboard.cpp	Dibujado y presentación visual de información (puntos, high score, nivel, vidas).
------------	----------------	---

La Clase AlienInvasion (Ejecutor)

En C++, debe ser una clase con encapsulamiento estricto (private para los métodos de control).

- **Control de Memoria:** En C++ el constructor debe reservar espacio en memoria y el destructor debe asegurar que cada puntero del vector de aliens sea liberado para evitar fugas (memory leaks).
- **El Bucle Principal:** En C++, esto requiere definir un booleano de control `bool running = true`; para evaluar todas las condiciones requeridas para inicio del juego.

La Clase Settings

En C++, esto puede ser implementado como un 'struct' o una clase inmutable, pasada por referencia constante a todas las demás clases para evitar copias innecesarias de memoria.

- **Tipado Estricto:** En C++ se debe definir el tipo explícitamente: `const float bulletSpeed = 1.5f`. Este parámetro de velocidad será valorado en la ejecución preliminar del juego.

Entidades: Ship, Alien, Bullet (Polimorfismo)

- **Herencia y Polimorfismo:** En C++, para que la clase ejecutora pueda actualizar todos los objetos de forma genérica, creemos que deberíamos definir una clase base, digamos `GameObject`, con un método virtual `void update() = 0`;
- Se requiere una jerarquía de clases estricta. Si `Ship` o `Alien` no implementan la interfaz `GameObject`, el compilador emitirá un error probablemente.

Lógica de Colisiones (Matemáticas de Rectángulos)

- **Matemática en C++:** Esto se traduce en una función lógica que compara los límites del rectángulo de la bala contra los de la nave alienígena. Apelaremos al mismo mecanismo utilizado en el Desafío 1 asociado con la construcción de un tetris.

JERARQUIA DE DEPENDENCIA

De acuerdo las descripciones dadas anteriormente, la clase **AlienInvasion** estará definida a partir de punteros o instancias que definirán las naves, el movimiento de estas y el tablero o telón de fondo, mientras que las clases hijas recibirán referencias a sus dependencias.

- **La Clase Dominante: AlienInvasion**

Es la única clase que conoce a todas las demás. Su responsabilidad es el Ciclo de Vida. Esta estaría dada por lo siguientes elementos:

- **Ship:** La instancia única del jugador.
- **GameStats:** El estado del juego (puntos, nivel, vidas).
- **Scoreboard:** Objeto visual que lee GameStats.
- **Button:** Objeto para controlar el estado.
- **std::vector<Alien>:** La flota de enemigos.
- **std::vector<Bullet>:** Los proyectiles.

- **La Dependencia Universal: Settings**

Esta es una clase de datos.

- **Porqué:** Casi todos los objetos (Ship, Alien, Bullet) necesitan conocer parámetros como `screen_width`, `bullet_speed` o `alien_color`. En lugar de que cada objeto copie estos datos, establecemos una referencia constante (const Settings) a través del constructor de cada clase. Esto garantiza que todos operen bajo las mismas reglas físicas del sistema.

El Grafo de Dependencias

Para visualizarlo mejor, el grupo imagina el flujo de dependencias como un grafo dirigido:

- **AlienInvasion** → (gestiona) → Ship, Alien (vector), Bullet (vector), Scoreboard, Button.
- **AlienInvasion** → (suministra informacion) → Settings a todas las anteriores.
- **Scoreboard** → (depende de) → GameStats (para leer datos).

- ```

classDiagram
 class GameObject {
 <<Abstract>>
 #x: float
 #y: float
 #speed: float
 #collider: Rect
 +virtual void update() = 0
 +virtual ~GameObject()
 }
 class Ship {
 -settings: const Settings&
 +Ship(const Settings&)
 +void update() override
 +void center_ship()
 }
 class Alien {
 -settings: const Settings&
 +Alien(const Settings&)
 +void update() override
 +bool check_edges()
 }
 class AlienInvasion {
 -running: bool
 -settings: Settings
 -stats: GameStats
 -sb: Scoreboard
 -ship: Ship
 -aliens: std::vector<Alien>
 -bullets: std::vector<Bullet>
 +run_game()
 +_check_events()
 +_update_aliens()
 +_update_bullets()
 }
 class Settings {
 centralRepository
 +const float screen_width
 +const float screen_height
 +const float bullet_speed
 +const float alien_speed
 }
 class GameStats {
 Estado
 +int score
 +int level
 +int ships_left
 }
 class Scoreboard {
 Interfaz
 -GameStats& stats
 +void show_score()
 }
 class Bullet {
 -settings: const Settings&
 +Bullet(const Settings&, float x, float y)
 +void update() override
 }
 class vector_Alien {
 vector<Alien>
 }
 class vector_Bullet {
 vector<Bullet>
 }

 GameObject <|-- Ship
 GameObject <|-- Alien
 Ship "1" -- "1" AlienInvasion
 Alien "1" -- "1" AlienInvasion
 AlienInvasion "1" *-- "1:N" vector_Alien
 AlienInvasion "1" *-- "1:N" vector_Bullet
 AlienInvasion "1" -- "1:N" Settings
 AlienInvasion "1" -- "1:N" GameStats
 AlienInvasion "1" -- "1:N" Scoreboard
 Settings <|-- Settings_centralRepository
 GameStats <|-- GameStats_estado
 Scoreboard <|-- Scoreboard_interfaz
 Bullet "1" -- "1:N" Settings

```



**UdeA**